Lecture 10 - An Example of Modern OCC

Background

- Paper: Speedy Transactions in Multicore In-Memory Databases, 2013

OLTP vs. OLAP

- In the 90s people started realizing that it is helpful to separate out specialized systems for OLTP from OLAP, rather than having one general-purpose system

OLTP

- OLTP: On-Line Transaction Processing. Very old term.
- Sometimes called “Operational Databases”.
- Textbook transactions as invented by Jim Gray.
- Typically, simple short-running queries with some updates.
- Sometimes OLTP tasks are pre-“canned”
  - (SQL “stored procedures” or as our reading says, “one-shot” tasks) because they’re just parameterized forms from applications, so the only variation is in the parameters.
  - TPC-C is the classic benchmark.

OLAP

- Originally just called “Decision Support” or “Analytics”.
- Associated with “Data Warehouses”, rather than “Operational Databases”.
  - Long-running, often complex, read-mostly query workloads with lots of aggregation (statistics).
- The term OLAP originally had a narrower meaning focusing on **Data Cubes**.
- Coined in the early 1990s by one Edgar F Codd, working as a consultant helping to market a product called Essbase.
- The pitch was supposed to be “revolutionary”, with a focus on “multidimensional analysis” – some attributes are “dimensions” (axes) and some are “measures”, forming a “data cube” to be sliced and diced.
- This narrower flavor of OLAP was the topic of some good research in the 1990s.
- Mostly reduced to SQL features by now (with extensions like CUBE BY), but some vestiges still exist of the Essbase-style pitch – e.g. MDX is a “standard” language pushed by Microsoft.
- A Data Cube is basically a hierarchy of GROUP BY queries. E.g. if you three dimensions like (make, model, color), you might want
  - sum-of-sales (no GROUP BY)
  - sum-of-sales GROUP BY make
  - sum-of-sales GROUP BY model
  - ...
  - sum-of-sales GROUP BY make, model, color

Comparisons

- Challenging to design/tune a DBMS to do well at both OLTP and OLAP.
- Problems around the mix of long- and short-running queries, plus
  - the mix of isolation-level requirements for the two workloads
- Conventional wisdom is to buy and/or deploy separate instances for OLTP (“operational DBMSs”) and for OLAP (“data warehouse”).

    - Different OLTP databases tend to “belong” to different applications/devops groups,
    - whereas DW tends to span all the data and “belong” to cross-cutting “decision makers” – e.g. management/“data science” team

Main-memory OLTP

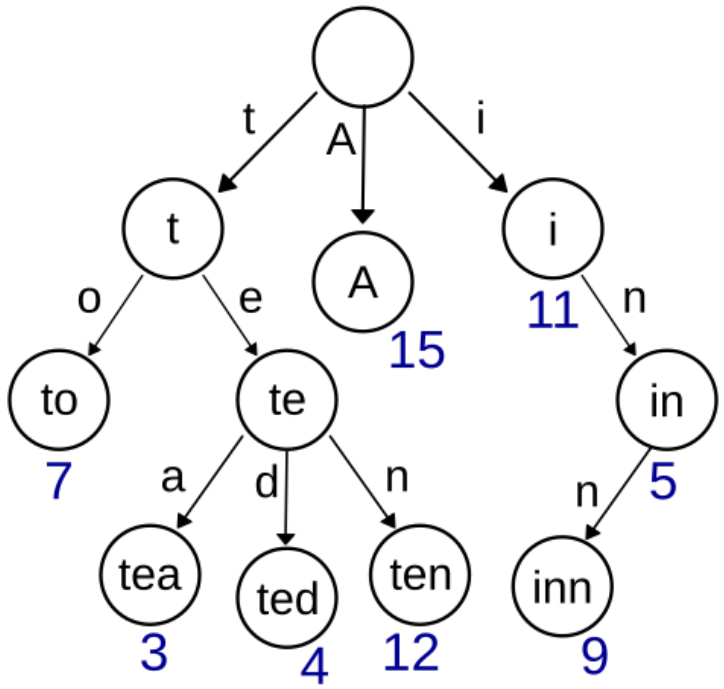
- Conventional wisdom
    - Disk is the primary storage medium
    - Memory is a buffer cache
    - Query processing must minimize disk I/O
- In the 2000s
    - Commodity servers had tens or hundreds of GB of RAM
    - Most OLTP data (e.g., TPC-C scale) could *fit entirely in memory*
    - You still want to write stuff out to disk for durability
- One attempt:
    - Stonebraker et al’s H-Store system, commercialized as VoltDB
        - still around as "Volt Active Data"
    - Shard DBMS across cores, hope that transactions shard the same way
        - Works for TPC-C benchmark! (shard by “warehouse”)
        - If not, lock entire shards
        - Silo, in reaction, is not sharded – all cores access shared data.
    - All transactions specified as "stored procedures"
        - essentially predeclared
- Other notable systems:
    - Hekaton, now part of Microsoft SQL Server
        - Other reading
    - SAP HANA (But also HTAP)
    - HyPer (later became part of Tableau)

Some Indexing Talk

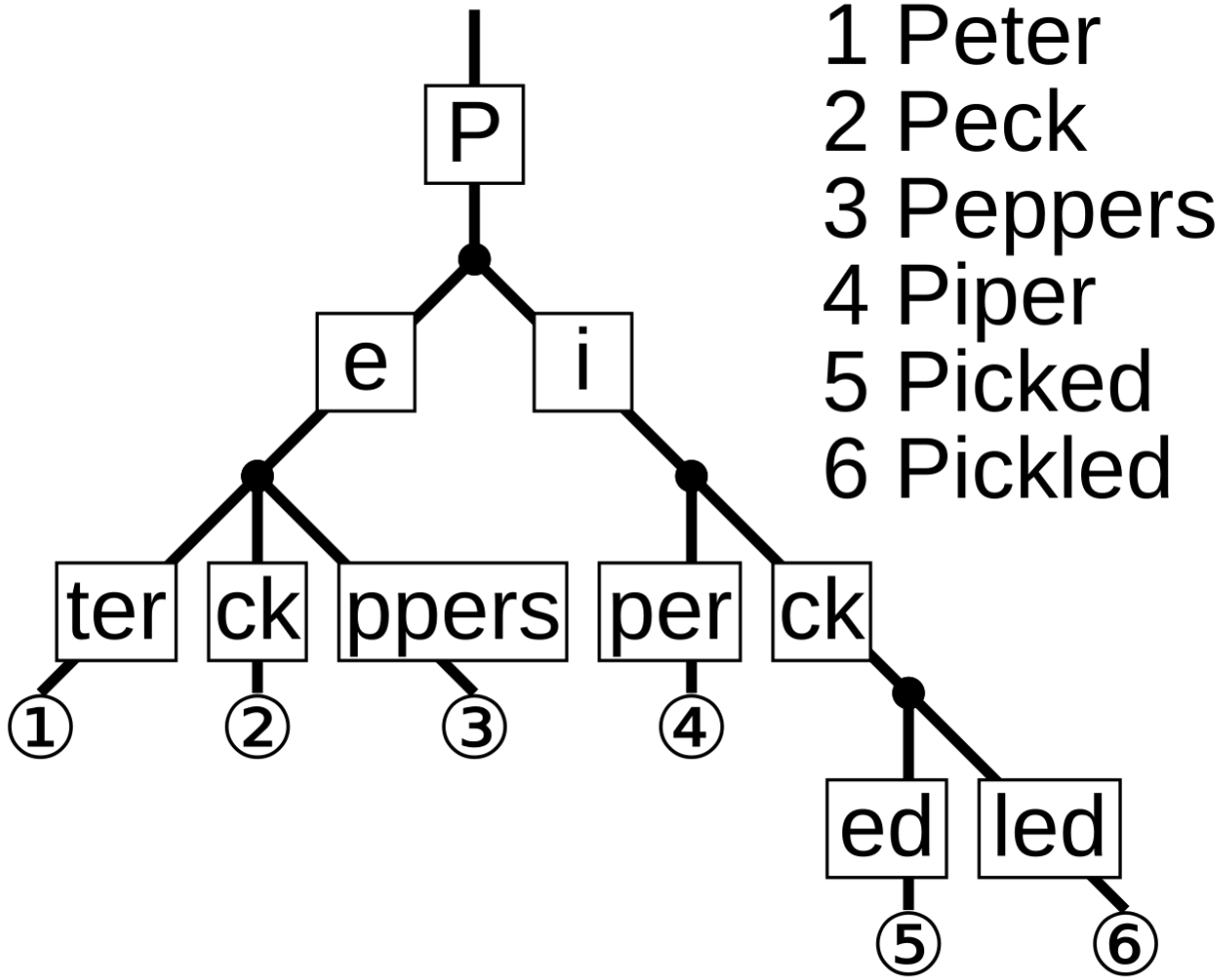
- The present system is based off of the Masstree in-memory KVS (so no txns)
    - tree/trie hybrid
    - “lock-free”, but makes use of atomics (test-and-set) for mutual exclusion
        - no better than locks under contention
            - could be maybe worse
            - some benefits in the absence of contention: no storage/compute overhead for locks
            - see discussion in Faleiro/Abadi CIDR 2017

Detour: Tries/Trees and Masstrees

- Not really essential to our reading but good to know
- Also known as prefix trees
    - A data structure that works well for storing variable length keys



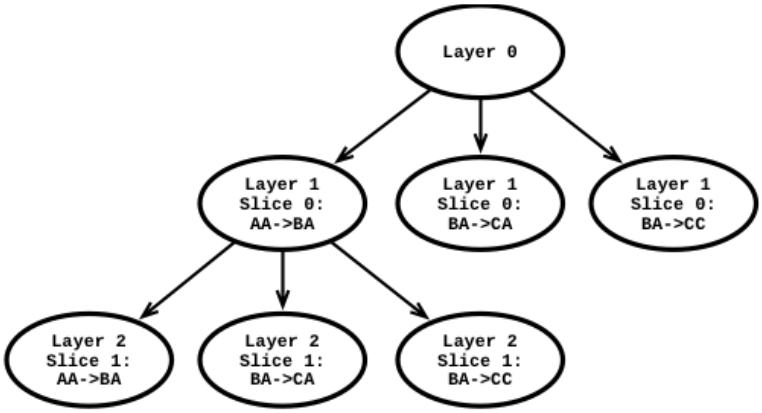
- Optimization *radix trees*
  - merge any solo child with its parent



- Use-cases:
  - Obvious: autocomplete
  - Key-value stores (especially efficient lookup in memory)

    - IP routing

What's right and wrong with B+Trees

- Optimized for disk, not really for memory
    - Low depth, high fan out
    - Don't need to retrieve too many pages (I/O cost), but computation is "free"
- But in-memory computation costs dominate
    - If keys are long, and share a long prefix, a B+tree will require repeated comparisons across all keys to find the right child to go down to
        - Cache inefficient because the comparisons make it non-deterministic
    - Worse if keys are variable length
- Idea behind Masstree: outer loop tries, inner loop B+trees
    - Outer loop divides up keys into slices, e.g., 8 chars
    - Per slice, have a B+tree
        - and don't have to redo comparisons for the prefix of the slice
    - 

    - (image from the paper trail)
- An alternative: Adaptive Radix Tree

Setup

Silo's goals

- In-memory DBMS
- Serializable isolation
- Multicore with shared memory
    - explicitly want to avoid partitioning as a "solution" since it won't work for cross-partition txns
    - also test a partitioned version
- We "*avoid all centralized contention points and make all synchronization scale with the data*"

First big idea: OCC without issues

- Want to use OCC rather than locking
    - OCC makes reads/writes in thread-local storage
    - Validation: if no concurrent txns writes overlapped with its read set, can install all the writes together
        - Due to validation, no need to lock read-set records
- However, need to use a central TID/communication across active txns —
    - or: track anti-dependencies for the purpose of logging and recovery

        - T1: R(A=10)
        - T2: W(A=20)
        - Here, T2 must be serialized after T1, so we need a mechanism to communicate the anti-dependency.
- Silo wants to avoid all shared-memory writes for read txns
    - "*carefully designed using memory fences to scalably produce results consistent with a serial order*"

Second big idea: epochs

- correct recovery solved using epoch-based group commit
- divide time into epochs
- relatively infrequent global “epoch counter” rather than transaction counter
- serializable, but system is only aware of this across epochs
    - within an epoch there may be txns with same ID (but they won't interfere, we promise!)
- results released to clients at epoch boundaries
- side benefit:
    - can use epochs to provide database snapshots for long running read txns
    - called *snapshot txns*; never abort

Architecture:

- “one-shot” tasks (stored procedures, all parameters known at time of request)
    - No data dependency on external world (user)
    - No user “think time” between BEGIN and COMMIT
    - Example: `transfer(src, dst, amount)`
- One worker thread / task -> assigned to a core
- Primary copy of data is in memory, but txns are made durable via logging to stable storage
    - results returned to user after logging

Transaction Protocol: Epochs

- Centers on epochs for serializability, several benefits:
    - they form natural serialization pts
    - garbage colletion
    - read-only snapshots
- Global Epoch number E in shared memory, updated every ~40ms, i.e. 25x per second
    - Set to be rare compared to txn duration
    - Incremented by a dedicated thread, no locking.
- Each worker w maintains a local epoch number e_w s.t. it is equal to E or $E-1$
    - Epoch-advancing thread must wait on lagging workers; E won't be incremented
    - Possible concern for mixed workloads!

TIDs and Data

- Unusual TIDs: 64 bits containing…
    - | E | X | status | |—|—|—|
    - **E** is the E at commit time
    - **X** is a unique ID per **E**
    - 3 **status** bits:
    - The tricky one is **X**. Paper says:

> A worker chooses a transaction's TID only after verifying that the transaction can commit.
> At that point, it calculates the smallest number that is
> (a) larger than the TID of any record read or written by the transaction, (b) larger than the worker's most recently chosen TID, and
> (c) in the current global epoch.
> The result is written into each record modified by the transaction.

  - **X** collisions may not be avoided!
- *TID order may not reflect serial order, unlike typical TO-based approaches!*
  - Anti-dependency: If t1 merely observed a tuple that t2 later overwrote, t1's TID might be either less than or greater than t2's!
  - Not a problem here (TODO: why?)
- Guarantees:
  - Each worker assigns monotonically increasing **X** per epoch, match serial order
  - TIDs per record monotonically increasing, match serial order
  - TIDs with different epochs match serial order
- Status bits
  - **lock**: an exclusive lock
    - helps co-locate locks with data (not new; TODO: where before?)
    - lock is closer to a latch (a low-level lock for memory structures)
  - **latest-version**: is this the latest version of this tuple
    - old versions can help for snapshot txns
  - **absent**: a "tombstone" that this tuple is deleted
- Tuple storage:
  - | TID | Previous Ptr | Data |
    - Data in place if small for cache locality
    - Update in place if possible (doesn't grow field size)

Commit Protocol

- Per worker, maintains a *read-set*, and *write-set* (new state of record)
  - Assumption: no *blind writes* (must read before writing!)
  - therefore, *write-set* $\subseteq$ *read-set*
- Phase 1
  - lock all tuples in *write-set* for transaction
    - deadlock avoidance: global order on tuples, acquire locks in this order.
      - Q: why is this OK? why can't we use this in other settings? A: one-shot txns, know all locks in advance
    - compiler_fence()
  - snapshot E to e_w // serialization point
    - compiler_fence()
- Phase 2
  - Re-fetch and compare each record in *read-set*
    - Abort if TID changed, latest bit = 0, or lock bit = 1 // TODO: why three conditions?
  - Assign a TID with $E = e_w$
- Phase 3
  - Write modified records to index with updated TID and set lock = 0, record-by-record
  - Note that TID and lock are sharing a memory word so change together atomically
- Note on compiler_fence: this is "global coordination", but on x86 and other "Total Store Order" machines, it's "free" … just that the compiler mustn't reorder code across the fence!

Proofs

- Serializability proof: by reduction to Strict 2PL (S2PL)
  - Recall strict 2PL acquires read/write locks in advance of access, drops all atomically at commit
  - Assume no blind writes, so *write-set* ⊆ *read-set*
  - *read-set* tuple: in Phase 2 we verify it's both unchanged & unlocked.
    - Hence we would have successfully acquired read lock in S2PL in advance of access, held it till commit point.
  - *write-set* tuple: would have successfully acquired/held S lock in S2PL, *and* upgraded to write lock successfully at commit time.
    - Recall: S lock check in Phase 2, but also X lock acquisition in Phase 1!
- Note: Epoch order conforms to serial order
  - compiler-fence before fetching E ensures our reads are before any future epoch happens
  - compiler-fence after write ensure transactions in future epoch would observe "at least" the lock bits from Phase 1
    - (or our committed records from this epoch)
- Let's take a couple of examples to convince ourselves this works:
  - Write skew
    - Here, two txns read same (union of) data, update different data
    - Txns could possibly acquire write locks on each in Phase 1, but whichever one gets to Phase 2 first will see a write lock on something it read (so abort), or no write lock, in which case it can commit having seen correct data
  - Example in paper: (x = y = 0)
    - t1 ← read(x); write(y ← t1 + 1)
    - t2 ← read(y); write(x ← t2 + 1)
    - For x = y = 1 to occur, both must read old values, and both must commit.
      - Suppose thread 1 reads x=0 and commits; then it verified that x was unlocked and unchanged
      - Since thread 2 locks x as part of phase 1, it has to do its serialization point after thread 1
        - it will either see thread 1's lock on y (has to have happened before its ser. pt.) or a new version
      - will set y = 2 or abort
- Why do updates work?
  - Concern: update-in-place, concurrent readers might have cached old versions
  - Approach:
    - first update the record while holding the lock
    - memory fence (TODO: what is this)
      - atomically expose new TID and released lock together
  - Hence concurrent reader (at start of txn) sees either a lock=1, or new data/TID
    - if lock=1, spin until lock = 0

Phantoms

- In addition to *read-set*, maintain *node-set*, use an idea that looks like *next value locking*, but without the explicit locks
- Hand-waved last time, mentioned Lomet'93 paper that allows locking of ranges and individual tuples
- Older paper that introduces *next-value locking* from Mohan: ARIES/KVL (VLDB'90)
  - B+ trees are used to lock ranges, not values
  - Suppose values in tree are 10, 20, 25, 30.
  - A txn reading 20 will lock the range (10,20]
  - ***a lock on a key value stands for locking the gap before it***
  - The Lomet'93 paper I mentioned is a generalization
- Another important issue: logging when performing index operations
  - Suppose a txn insertion leads to a B+tree update (e.g., split of node), but then is aborted
  - Undoing the B+tree update could be very expensive and, potentially incorrect because other txns may depend on it

        - example:
            - T1: insert -> split page
            - T2: reads new page
    - Idea is that structural operations are logically separate; a split is a physical consequence
        - One can undo the insert for T1, keep the structural change
    - Needs **nested top actions**. A sub-operation within a txn that:
        - Executes *within* a transaction
        - Commits independently
        - **Will never be undone**, even if the parent transaction aborts.
- Silo basically implicitly obtains read locks on all pertinent nodes of the B+tree without actually doing so
    - Checks if the node versions have changed during validation

Snapshot Txns

- Snapshot epochs are similar to regular epochs, but advance more slowly
- When modifying, check if the snapshot epoch of the txn one is about to edit is same as current snapshot, then safe to overwrite,
- Otherwise make another copy, and have a back-ptr
- As snapshot epochs advance, these back-ptrs become stale and less relevant

Logging/Durability

- Treats whole epochs as durability units
    - *The results of transactions in epoch e are not released to clients until all transactions with epochs≤ e have been stored durably.*
- Commit entire epochs to disk. Atomic epochs, *no prefix of epoch*!
  - because serial order within epoch is not recoverable from the log! - maybe this is why it's OK for 2 workers to choose the same **X**?
- Log record for a transaction: TID, list of (table, key, value) for modified records
- record-level physical REDO logging.
    - No UNDO since they log after txns commit,
    - No logical (operational) logging.

Evaluation

- SILO did well!